Q.1) Write a C Program that demonstrates redirection of standard output to a file

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
int main() {
int fd;
// Open (or create) the output file
fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd < 0) {
perror("open");
exit(1);
}
// Redirect stdout to file
if (dup2(fd, STDOUT_FILENO) < 0) {
perror("dup2");
exit(1);
}
close(fd); // Close the original file descriptor
// Now all printf output goes to output.txt
printf("This output is redirected to a file.\n");
printf("Hello, this is C programming redirection demo!\n");
return 0;
}
```

Q.2) Implement the following unix/linux command (use fork, pipe and exec system call)

ls –l | wc –l

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
int fd[2];
pid_t pid1, pid2;
// Create a pipe
if (pipe(fd) == -1) {
perror("pipe");
exit(1);
}
// First child: execute "ls -l"
pid1 = fork();
if (pid1 < 0) {
perror("fork");
exit(1);
}
if (pid1 == 0) {
```

```
// Child 1 process
close(fd[0]); // Close unused read end
dup2(fd[1], STDOUT_FILENO); // Redirect stdout to pipe
close(fd[1]);
execlp("ls", "ls", "-l", NULL);
perror("execlp ls");
exit(1);
}
// Second child: execute "wc -l"
pid2 = fork();
if (pid2 < 0) {
perror("fork");
exit(1);
}
if (pid2 == 0) {
// Child 2 process
close(fd[1]); // Close unused write end
dup2(fd[0], STDIN_FILENO); // Redirect stdin from pipe
close(fd[0]);
execlp("wc", "wc", "-l", NULL);
perror("execlp wc");
exit(1);
}
// Parent process
close(fd[0]);
close(fd[1]);
// Wait for both children to finish
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
return 0;
}
```